Bifurcation

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters of your map
a = 0.9
c = 0.15

# Range of control parameter b
b_min, b_max = 6.1, 10
b_points = 800  # resolution in b

# Iteration settings
iterations = 1500
transient = 1000
x0 = 0.2345  # initial condition

# Generate bifurcation data
b_values = np.linspace(b_min, b_max, b_points)
x_list, b_list = [], []

for b in b_values:
    x = x0
    for i in range(iterations):
        # Map definition
        x = (a * np.sin(2 * np.pi * x)
             + b * (x - 0.5)**3
             + c * x) % 1.0

        # After transient, record data
        if i >= transient:
            x_list.append(x)
            b_list.append(b)

# Plot bifurcation diagram
plt.figure(figsize=(10, 5))
plt.plot(b_list, x_list, ',k', alpha=0.25)
plt.title(f"Bifurcation diagram of custom map\n(a={a}, c={c})")
plt.xlabel(r"$b$")
plt.ylabel(r"$x$")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Lyapunov exponent

```
# -- coding: utf-8 --
"""
Created on Mon Nov 17 10:33:11 2025

@author: Noor_1
"""

# lyapunov_custom_map.py
import numpy as np
import math
import matplotlib.pyplot as plt

# -------------------------
# Parameters of your map
# -------------------------
a = 0.9
c = 0.15
x0 = 0.321

# b-range and sampling
b_min, b_max = 6.1, 10.0
b_points = 600

# iteration settings for Lyapunov
transient = 800
n_iter = 1200
eps = 1e-12

# -------------------------
# Map and derivative
# -------------------------
def map_step(x, b):
    """Your map: F(x) = a*sin(2πx) + b(x−0.5)^3 + cx"""
    val = (a * math.sin(2.0 * math.pi * x)
           + b * (x - 0.5)**3
           + c * x)
    return val % 1.0

def derivative(x, b):
    """Analytic derivative dF/dx"""
    term1 = a * (2.0 * math.pi) * math.cos(2.0 * math.pi * x)
    term2 = b * 3.0 * (x - 0.5)**2
```

```
    term3 = c
    return term1 + term2 + term3

# -------------------------
# Sweep b and compute Lyapunov
# -------------------------
b_values = np.linspace(b_min, b_max, b_points)
lyap_values = np.empty_like(b_values)

for i, b in enumerate(b_values):
    x = x0

    # transient
    for _ in range(transient):
        x = map_step(x, b)

    # accumulate Lyapunov sum
    lyap_sum = 0.0
    count = 0

    for _ in range(n_iter):
        d = derivative(x, b)

        if not np.isfinite(d) or abs(d) < 1e-14:
            d = 1e-14  # avoid log(0)

        lyap_sum += math.log(abs(d) + eps)
        count += 1

        x = map_step(x, b)
        if not np.isfinite(x):
            break

    lyap_values[i] = lyap_sum / count if count > 0 else float('nan')

# -------------------------
# Identify chaotic intervals (λ > 0)
# -------------------------
threshold = 0.0
is_chaotic = lyap_values > threshold
intervals = []
start_idx = None

for idx, flag in enumerate(is_chaotic):
    if flag and start_idx is None:
        start_idx = idx
    if (not flag or idx == len(is_chaotic)-1) and start_idx is not None:
        end_idx = idx if not flag else idx
```

```
        intervals.append((b_values[start_idx], b_values[end_idx]))
        start_idx = None

# -------------------------
# Print intervals and summary
# -------------------------
print("Chaotic b-intervals (Lyapunov > 0):")
for (p, q) in intervals:
    print(f"  b ∈ [{p:.6f}, {q:.6f}]   (width = {q - p:.6f})")

mean_lyap = np.nanmean(lyap_values)
max_lyap = np.nanmax(lyap_values)
print(f"\nLyapunov summary: mean λ = {mean_lyap:.6f}, max λ = {max_lyap:.6f}")

# -------------------------
# Plot Lyapunov spectrum
# -------------------------
plt.figure(figsize=(10, 4))
plt.plot(b_values, lyap_values, '-', linewidth=1.0)
plt.axhline(0.0, linestyle='--', color='k', linewidth=1)

# shade chaotic regions
for (p, q) in intervals:
    plt.axvspan(p, q, alpha=0.12, color='red')

plt.xlabel('b')
plt.ylabel('Lyapunov exponent λ')
plt.title(f'Lyapunov spectrum λ(b) for custom map (a={a}, c={c})')
plt.grid(alpha=0.3)
plt.tight_layout()

plt.savefig('lyapunov_custom_map.png', dpi=300)
plt.show()

# save numeric results
np.savetxt('b_vs_lyap.txt',
           np.column_stack((b_values, lyap_values)),
           header='b lyap', fmt='%.8f')

print("\nSaved: lyapunov_custom_map.png and b_vs_lyap.txt")
```

—----------

Fuzzy entropy test

```
# -*- coding: utf-8 -*-
"""
Created on Mon Nov 17 10:41:05 2025

@author: Noor_1
"""

import numpy as np
import math
import matplotlib.pyplot as plt

# -------------------------
# Map parameters (fixed)
# -------------------------
a = 0.9
c = 0.15

# -------------------------
# User settings (change if needed)
# -------------------------
b_values = np.linspace(6.1, 10.0, 40)    # sweep over parameter b
x0 = 0.321                               # initial condition
traj_length = 2000                       # samples AFTER burn-in
transient = 1500                         # burn-in iterations
m_embed = 2
r_factor = 0.2
max_vectors = 2000
savefile = "fuzzy_entropy_vs_b_chaotic_map.png"
show_annotation = True

# -------------------------
# Your Map (one iteration)
# -------------------------
def chaotic_step(x, b_control):
    val = (
        a * math.sin(2 * math.pi * x) +
        b_control * (x - 0.5)**3 +
        c * x
    )
    return val % 1.0

# -------------------------
# Generate trajectory (with burn-in)
# -------------------------
def generate_trajectory(b_control, x0=x0, length=traj_length, transient=transient):
```

```
    total = length + transient
    x = float(x0)
    out = []
    for i in range(total):
        x = chaotic_step(x, b_control)
        if i >= transient:
            out.append(x)
    return np.array(out, dtype=float)

# -------------------------
# Build embedded vectors
# -------------------------
def embedded_vectors(x, m):
    N = len(x)
    if N - m + 1 <= 0:
        return np.empty((0, m))
    return np.vstack([x[i:i+m] for i in range(N - m + 1)])

# -------------------------
# Fuzzy Entropy (Gaussian membership)
# -------------------------
def fuzzy_entropy(signal, m=2, r_factor=0.2, max_vectors=2000):
    x = np.asarray(signal, dtype=float)
    N = len(x)
    if N < (m + 2):
        return float('nan')

    r_tol = r_factor * np.std(x)
    if r_tol == 0:
        r_tol = 1e-8

    X_m = embedded_vectors(x, m)
    X_m1 = embedded_vectors(x, m + 1)

    def avg_similarity(X):
        P = X.shape[0]
        if P <= 1:
            return 0.0
        if (max_vectors is not None) and (P > max_vectors):
            rng = np.random.default_rng(12345)
            idxs = rng.choice(P, size=max_vectors, replace=False)
            Xs = X[idxs]
        else:
            Xs = X

        P2 = Xs.shape[0]
        Phi_sum = 0.0
        for i in range(P2):
```

```
            xi = Xs[i]
            dists = np.max(np.abs(Xs - xi), axis=1)
            mask = np.ones(P2, dtype=bool)
            mask[i] = False
            d_others = dists[mask]
            mu = np.exp(- (d_others ** 2) / (r_tol + 1e-30))
            Phi_sum += np.mean(mu)
        return Phi_sum / P2

    Phi_m = avg_similarity(X_m)
    Phi_m1 = avg_similarity(X_m1)

    if Phi_m <= 0 or Phi_m1 <= 0:
        return float('nan')

    return -math.log(Phi_m1 / Phi_m)

# -------------------------
# Sweep b and compute FuzzyEn
# -------------------------
fe_values = np.empty_like(b_values, dtype=float)
for i, b in enumerate(b_values):
    traj = generate_trajectory(b_control=b)
    fe = fuzzy_entropy(traj, m=m_embed, r_factor=r_factor, max_vectors=max_vectors)
    fe_values[i] = fe

# -------------------------
# Plot
# -------------------------
plt.figure(figsize=(10, 4.5))
plt.plot(b_values, fe_values, marker='o', linestyle='-')

if show_annotation and np.any(np.isfinite(fe_values)):
    idx = int(np.nanargmax(fe_values))
    b_max = float(b_values[idx])
    fe_max = float(fe_values[idx])
    plt.plot(b_max, fe_max, 'ro')
    plt.annotate(f"b={b_max:.4f}\nFuzzyEn={fe_max:.4f}",
                 xy=(b_max, fe_max),
                 xytext=(b_max + 0.1, fe_max),
                 arrowprops=dict(arrowstyle='->', lw=0.8),
                 bbox=dict(boxstyle='round', fc='w', alpha=0.8))

plt.xlabel('b (control parameter)')
plt.ylabel('Fuzzy Entropy')
plt.title('Fuzzy Entropy vs b (Chaotic Map)')
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.savefig(savefile, dpi=220)
plt.show()
```

Shanon entropy code

```
# -*- coding: utf-8 -*-
"""
Created on Mon Nov 17 10:55:20 2025

@author: Noor_1
"""
# shannon_entropy_custom_map.py
import numpy as np
import math
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# Shannon Entropy Calculation for Your Chaotic Map
# ------------------------------------------------------------
def chaotic_map_entropy(b, a=0.9, c=0.15,
                        x0=0.612, N=200000, burn_in=1000, F=1024):

    x = x0
    counts = np.zeros(F)

    # Burn-in: remove transient effects
    for _ in range(burn_in):
        x = (a * np.sin(2 * np.pi * x)
             + b * (x - 0.5)**3
             + c * x) % 1.0

    # Collect data
    for _ in range(N):
        x = (a * np.sin(2 * np.pi * x)
             + b * (x - 0.5)**3
             + c * x) % 1.0

        index = int(x * F)
        if index >= F:
            index = F - 1
        counts[index] += 1

    # Convert counts to probabilities
    P = counts / N
```

```
    # Shannon entropy (bits)
    H = -np.sum([p * math.log(p, 2) for p in P if p > 0])
    return H


# ----------------------------------------------------------
# Sweep b values and compute entropy
# ----------------------------------------------------------
b_min = 6.1
b_max = 10.0
num_points = 100   # increase for smoother curve

b_values = np.linspace(b_min, b_max, num_points)
entropy_values = []

print("Computing Shannon entropy for your map...\n")
for b in b_values:
    H = chaotic_map_entropy(b)
    entropy_values.append(H)
    print(f"b = {b:.4f} --> H = {H:.5f} bits")

# ----------------------------------------------------------
# Find Max & Min Entropy
# ----------------------------------------------------------
max_H = max(entropy_values)
min_H = min(entropy_values)

b_at_max = b_values[entropy_values.index(max_H)]
b_at_min = b_values[entropy_values.index(min_H)]

print("\n-------------------------------------------")
print(f"Maximum Shannon Entropy : {max_H:.5f} bits at b = {b_at_max:.4f}")
print(f"Minimum Shannon Entropy : {min_H:.5f} bits at b = {b_at_min:.4f}")
print("-------------------------------------------\n")

# ----------------------------------------------------------
# Plot Shannon Entropy vs b
# ----------------------------------------------------------
plt.figure(figsize=(8, 5))
plt.plot(b_values, entropy_values, marker='o', linewidth=1)
plt.xlabel("Control Parameter b")
plt.ylabel("Shannon Entropy (bits)")
plt.title("Shannon Entropy vs b for Custom Chaotic Map")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Correlation test for a choatic map

```
import numpy as np
import math
import matplotlib.pyplot as plt

# -------------------------
# Parameters of your map
# -------------------------
a = 0.9
c = 0.15

# -------------------------
# Your chaotic map
# -------------------------
def map_step(x, b):
    val = (
        a * math.sin(2.0 * math.pi * x)
        + b * (x - 0.5)**3
        + c * x
    )
    return val % 1.0

# -------------------------
# Pearson correlation
# -------------------------
def correlation(x_seq, y_seq):
    x = np.asarray(x_seq, dtype=float)
    y = np.asarray(y_seq, dtype=float)
    xm = x.mean()
    ym = y.mean()
    num = np.mean((x - xm) * (y - ym))
    den = x.std(ddof=0) * y.std(ddof=0)
    if den == 0:
        return 0.0
    return num / den

# -------------------------
# Generate sequence
# -------------------------
def generate_sequence(b, x0, length, transient):
    x = float(x0)

    # transient
```

```
    for _ in range(transient):
        x = map_step(x, b)

    seq = np.empty(length, dtype=float)
    for i in range(length):
        x = map_step(x, b)
        seq[i] = x

    return seq

# ------------------------
# Sweep b values
# ------------------------
b_min, b_max = 6.1, 10.0
num_b_samples = 200
b_values = np.linspace(b_min, b_max, num_b_samples)

# x0 sweep settings
x0_min, x0_max = 0.1, 0.9
num_x0_samples = 200
x0_values = np.linspace(x0_min, x0_max, num_x0_samples)

# Iteration parameters
sequence_length = 1024
transient = 256
perturbation = 1e-10

# choose chaotic b for x0 sweep
b_for_x0_sweep = 8.431

# Output files
save_fig_b = "corr_vs_b.png"
save_fig_x0 = "corr_vs_x0.png"
save_data_b = "correlation_vs_b.txt"
save_data_x0 = "correlation_vs_x0.txt"

# ------------------------
# Sweep 1: Correlation vs b
# ------------------------
corr_vs_b = np.empty_like(b_values, dtype=float)
base_x0 = 0.321

for i, b in enumerate(b_values):
    seq_a = generate_sequence(b, base_x0, sequence_length, transient)
    seq_b = generate_sequence(b, base_x0 + perturbation, sequence_length, transient)
    cr = correlation(seq_a, seq_b)
    corr_vs_b[i] = abs(cr)
```

```
# Save numeric results
np.savetxt(save_data_b, np.column_stack((b_values, corr_vs_b)),
           header="b abs_correlation", fmt="%.8e")

idx_b_max = int(np.nanargmax(corr_vs_b))
b_max_val = b_values[idx_b_max]
cr_max_val = corr_vs_b[idx_b_max]

# -------------------------
# Plot: Correlation vs b
# -------------------------
plt.figure(figsize=(10, 5))
plt.plot(b_values, corr_vs_b, marker='.', linestyle='-', markersize=4)
plt.axhline(0.0, linestyle='--', linewidth=0.8, color='gray')
plt.xlabel('b (control parameter)')
plt.ylabel('|C_b|')
plt.title('Absolute Correlation vs b (perturbation in $x_0$)')
plt.grid(alpha=0.3)
plt.plot(b_max_val, cr_max_val, 'ro')
plt.annotate(f"max |C_b|={cr_max_val:.6e}\nat b={b_max_val:.6f}",
             xy=(b_max_val, cr_max_val),
             xytext=(b_max_val + 0.03, cr_max_val),
             arrowprops=dict(arrowstyle="->", lw=0.7),
             bbox=dict(boxstyle="round", fc="w", alpha=0.8))
plt.tight_layout()
plt.savefig(save_fig_b, dpi=220)
plt.show()
plt.close()

# -------------------------
# Sweep 2: Correlation vs x0
# -------------------------
corr_vs_x0 = np.empty_like(x0_values, dtype=float)

for i, x0 in enumerate(x0_values):
    seq_a = generate_sequence(b_for_x0_sweep, x0, sequence_length, transient)
    seq_b = generate_sequence(b_for_x0_sweep, x0 + perturbation, sequence_length,
transient)
    cr = correlation(seq_a, seq_b)
    corr_vs_x0[i] = abs(cr)

# Save numeric results
np.savetxt(save_data_x0, np.column_stack((x0_values, corr_vs_x0)),
           header="x0 abs_correlation", fmt="%.8e")

idx_x0_max = int(np.nanargmax(corr_vs_x0))
x0_max_val = x0_values[idx_x0_max]
cr_x0_max_val = corr_vs_x0[idx_x0_max]
```

```
# -------------------------
# Plot: Correlation vs x0
# -------------------------
plt.figure(figsize=(10, 5))
plt.plot(x0_values, corr_vs_x0, marker='.', linestyle='-', markersize=4)
plt.axhline(0.0, linestyle='--', linewidth=0.8, color='gray')
plt.xlabel('$x_0$ (initial condition)')
plt.ylabel('|C_{x0}|')
plt.title(f'Absolute Correlation vs $x_0$ (b = {b_for_x0_sweep:.4f})')
plt.grid(alpha=0.3)
plt.plot(x0_max_val, cr_x0_max_val, 'ro')
plt.annotate(f"max |C|={cr_x0_max_val:.6e}\nat x0={x0_max_val:.6f}",
             xy=(x0_max_val, cr_x0_max_val),
             xytext=(x0_max_val + 0.03, cr_x0_max_val),
             arrowprops=dict(arrowstyle="->", lw=0.7),
             bbox=dict(boxstyle="round", fc="w", alpha=0.8))
plt.tight_layout()
plt.savefig(save_fig_x0, dpi=220)
plt.show()
plt.close()

# -------------------------
# Summary
# -------------------------
print("Correlation vs b: max |C_b| = {:.6e} at b = {:.6f}".format(cr_max_val, b_max_val))
print("Correlation vs x0: max |C| = {:.6e} at x0 = {:.6f} (b = {:.4f})".format(
    cr_x0_max_val, x0_max_val, b_for_x0_sweep))
print(f"Saved data: {save_data_b}, {save_data_x0}")
print(f"Saved figures: {save_fig_b}, {save_fig_x0}")
```

—----------

Average num of 1bit test

```
# -*- coding: utf-8 -*-
"""
Created on Mon Nov 17 11:19:18 2025

@author: Noor_1
"""

import numpy as np
import matplotlib.pyplot as plt

# --- Your Chaotic Map ---
def map_step(x, b, a=0.9, c=0.15):
```

```
    val = a * np.sin(2.0 * np.pi * x) + b * (x - 0.5)**3 + c * x
    return val % 1.0  # keep in [0,1]

# --- Generate chaotic sequence ---
def generate_sequence(b, x0=0.7, N=1000):
    seq = np.zeros(N)
    seq[0] = x0
    for i in range(1, N):
        seq[i] = map_step(seq[i-1], b)
    return seq

# --- Convert float to integer byte (0–255) ---
def float_to_8bit_int(x):
    return np.floor(x * 256).astype(int)

# --- Count number of 1 bits in an 8-bit integer ---
def count_ones_in_byte(n):
    return bin(n).count('1')

# --- b-range (control parameter) ---
b_values = np.arange(6.1, 10.01, 0.1)
average_ones = []

# --- Run test ---
for b in b_values:
    seq = generate_sequence(b)
    bytes_seq = float_to_8bit_int(seq)

    total_ones = sum(count_ones_in_byte(bx) for bx in bytes_seq)
    average_ones.append(total_ones)

# Ideal expected ones = (total bits)/2 = (1000 * 8)/2 = 4000
ideal_ones = 4000

# --- Plot results ---
plt.figure(figsize=(8,5))
plt.plot(b_values, average_ones, 'bo-', label='Average Number of 1s')
plt.axhline(y=ideal_ones, color='r', linestyle='--', label='Ideal value (4000)')
plt.title('Average Number of 1s vs b (Your Chaotic Map)')
plt.xlabel('b (control parameter)')
plt.ylabel('Number of 1s (out of 8000 bits)')
plt.legend()
plt.grid(True)
plt.show()

# --- Save results ---
with open("average_ones_vs_b.txt", "w") as f:
    f.write("b\tAverage_Number_of_1s\n")
```

```
    for b, ao in zip(b_values, average_ones):
        f.write(f"{b:.3f}\t{ao}\n")
```

---

bin file creation code for map

```
import numpy as np
import math
import time

# ============================================================
# Chaotic Map Base Parameters
# ============================================================
b_min, b_max = 6.1, 10.0     # b range for your map
a = 0.9
c = 0.15
x0 = 0.3456789               # Initial seed ∈ (0,1)

warmup_iters = 10000
nist_bits_per_file = 800_000_000   # 100 MB output
nist_values = nist_bits_per_file // 32
max_uint32 = np.uint64(2**32 - 1)

# ============================================================
# Chaotic Map F(x) = a sin(2πx) + b(x − 0.5)^3 + c x  mod 1
# ============================================================
def chaotic_map(x, b):
    """Your chaotic map"""
    xn1 = (
        a * math.sin(2 * math.pi * x) +
        b * (x - 0.5)**3 +
        c * x
    ) % 1.0
    return xn1

# ============================================================
# Whitening Function (bit diffusion)
# ============================================================
def whiten_uint32(arr):
    a2 = arr.copy()
    a2 ^= (a2 >> np.uint32(16))
    a2 ^= ((a2 << np.uint32(13)) & np.uint32(0xFFFFFFFF))
    a2 ^= (a2 >> np.uint32(5))
```

```
    tmp = (a2.astype(np.uint64) * np.uint64(0x9E3779B185EBCA87)) &
np.uint64(0xFFFFFFFFFFFFFFFF)
    tmp = (tmp + np.uint64(0xBB67AE8584CAA73B)) & np.uint64(0xFFFFFFFFFFFFFFFF)

    a2 = (tmp ^ (tmp >> np.uint64(32))).astype(np.uint32)
    return a2

# ============================================================
# Dynamic b(t) generation using chaotic feedback
# ============================================================
def randomize_b(x):
    """Generate pseudo-random b(t) ∈ [6.1, 10]"""
    return b_min + (b_max - b_min) * ((math.sin(2 * math.pi * x) + 1) / 2)

# ============================================================
# Start sequence generation
# ============================================================
print("Generating NIST bitstream (~100 MB)...")
print(f"Initial seed x0 = {x0}")
print(f"Parameter range: b ∈ [{b_min}, {b_max}], a={a}, c={c}")

# --- Warm-up (remove transient behavior) ---
x = float(x0)
for _ in range(warmup_iters):
    b = randomize_b(x)     # random dynamic b(t)
    x = chaotic_map(x, b)

# --- Main chaotic sequence ---
xs = np.empty(nist_values, dtype=np.float64)
xs[0] = x

for i in range(1, nist_values):
    b = randomize_b(xs[i-1])       # dynamic-chaotic b(t)
    xs[i] = chaotic_map(xs[i-1], b)

    if i % 1_000_000 == 0:
        print(f"Progress: {100*i/nist_values:.1f}%...", end="\r")

# ============================================================
# Convert to uint32 and whiten
# ============================================================
x_ints = np.floor(xs * float(max_uint32)).astype(np.uint32)

rolled = np.roll(x_ints, 1)
x_mix = np.bitwise_xor(x_ints, rolled)
x_white = whiten_uint32(x_mix)
```

```
# ============================================================
# Convert uint32 array → bit-level bytes
# ============================================================
def uint32_to_bitbytes(u32_array):
    bits = np.unpackbits(u32_array.view(np.uint8))
    return np.packbits(bits)

x_bytes = uint32_to_bitbytes(x_white)

# ============================================================
# Save file for NIST STS
# ============================================================
filename = f"chaosmap_b_dynamic_{int(time.time())}.nist.bin"
with open(filename, "wb") as f:
    f.write(x_bytes)

print("\n\n==========================================")
print("✅ NIST-compatible file generated successfully!")
print(f"📁 File: {filename}  (~100 MB)")
print("==========================================\n")

print("Run NIST STS as:")
print(f"  ./assess 100000000 {filename}")
```

—----------

#Invariant densities test of map

```
# -*- coding: utf-8 -*-
"""
Created on Tue Nov 18 05:33:33 2025

@author: Noor_1
"""

import math
import numpy as np
from numpy.random import default_rng
from scipy.stats import ks_2samp
import matplotlib.pyplot as plt

rng = default_rng(12345)

# ------------------------
# Custom map step (your map)
```

```
# F(x) = a*sin(2πx) + b*(x-0.5)^3 + c*x   (mod 1)
# ------------------------
def custom_step(x, params, k=None):
    """
    Single iteration of the user map. If k is provided, deep-zoom is applied.
    params: dict with keys 'b', 'a', 'c' (a,c optional if present in closure)
    """
    x = float(x) % 1.0
    b = params.get('b', 8.0)
    a = params.get('a', 0.9)
    c = params.get('c', 0.15)

    term1 = a * math.sin(2.0 * math.pi * x)
    term2 = b * (x - 0.5) ** 3
    term3 = c * x
    y = (term1 + term2 + term3) % 1.0

    if k is None or k <= 0:
        return y

    # deep zoom: fractional part of 10^k * y
    factor = pow(10.0, float(k))
    return (factor * y) % 1.0

# -------------------------
# deep_zoom utility (separate for clarity)
# -------------------------
def deep_zoom(x, k):
    if k is None or k <= 0:
        return float(x) % 1.0
    factor = pow(10.0, float(k))
    return (factor * float(x)) % 1.0

# -------------------------
# k-step wrapper (alternate interface)
# -------------------------
def k_custom_step(x, params, k):
    y = custom_step(x, params, k=None)
    return deep_zoom(y, k)

# -------------------------
# Generate trajectory
# -------------------------
def generate_trajectory(step_func, x0, params, n_iters, discard=0, k=None):
    """
    Generate a trajectory of length n_iters after discarding 'discard' transient iterations.
    step_func: function(x, params, k=None) -> next_x
    """
```

```
    x = float(x0) % 1.0
    traj = np.empty(n_iters, dtype=float)
    total = discard + n_iters
    for i in range(total):
        x = step_func(x, params, k=k)
        if i >= discard:
            traj[i - discard] = x
    return traj

# ------------------------
# Ergodicity helpers (time vs ensemble, KS test, etc.)
# ------------------------
def time_vs_ensemble(step_func, params, x0, k, T=20000, T0=2000, M=200):
    """
    Compare time averages vs ensemble averages for several observables.
    Returns results dict and one long trajectory (for diagnostics).
    """
    # indicator helper
    def I_interval(x, a, b):
        return ((x >= a) & (x < b)).astype(float)

    # generate long trajectory
    traj = generate_trajectory(step_func, x0, params, n_iters=T, discard=T0, k=k)

    obs = {
        'x': (lambda x: x),
        'x2': (lambda x: x**2),
        'I[0.25,0.75]': (lambda x: I_interval(x, 0.25, 0.75))
    }

    results = {}
    for name, f in obs.items():
        time_samples = f(traj)
        time_avg = float(np.mean(time_samples))

        # ensemble: pick M random initials and take last value after T iterations
        ens_samples = np.empty(M, dtype=float)
        initials = rng.uniform(0.1, 0.9, size=M)
        for i, xi in enumerate(initials):
            seq = generate_trajectory(step_func, xi, params, n_iters=T, discard=T0, k=k)
            ens_samples[i] = float(f(seq[-1]))  # take scalar observable at last iterate

        ks_D = float(ks_2samp(time_samples, ens_samples).statistic)
        results[name] = {
            'time_avg': time_avg,
            'ens_mean': float(np.mean(ens_samples)),
            'ens_std': float(np.std(ens_samples)),
            'ks_D': ks_D
```

```
        }
    return results, traj

# -------------------------
# histogram L1 difference between multiple initializations
# -------------------------
def hist_L1(traj_list, nbins=300):
    edges = np.linspace(0.0, 1.0, nbins + 1)
    hists = [np.histogram(tr, bins=edges, density=True)[0] for tr in traj_list]
    base = hists[0]
    # L1 approx: sum |base - h| * bin_width
    bin_width = 1.0 / nbins
    l1s = [float(np.sum(np.abs(base - h)) * bin_width) for h in hists[1:]]
    return edges, hists, l1s

# -------------------------
# autocorrelation and exponential fit (log fit) for early lags
# -------------------------
def autocorr_and_fit(traj, maxlag=2000):
    x = traj - np.mean(traj)
    n = len(x)
    ac = np.array([ np.dot(x[:n - k], x[k:]) / (n - k) for k in range(maxlag + 1) ])
    ac = ac / ac[0]
    # fit log(ac) vs lag for lags where ac > tiny threshold
    L = min(500, maxlag)
    mask = ac[1:L] > 1e-12
    if np.sum(mask) < 5:
        return ac, np.nan, np.nan
    lags = np.arange(1, L)[mask]
    y = np.log(ac[1:L][mask])
    slope, intercept = np.polyfit(lags, y, 1)  # y = slope*lag + intercept
    return ac, float(slope), float(intercept)

# -------------------------
# Benettin-style Lyapunov via finite-difference (wrap-aware)
# -------------------------
def lyapunov_fd(step_func, params, x0=0.321, k=None, T=50000, discard=5000,
delta=1e-9):
    x = float(x0) % 1.0
    y = (x + delta) % 1.0
    s = 0.0
    count = 0
    total = discard + T
    for n in range(total):
        x = step_func(x, params, k=k)
        y = step_func(y, params, k=k)
        # wrap-aware difference
        dx = ((y - x + 0.5) % 1.0) - 0.5
```

```
        dist = abs(dx)
        if dist < 1e-16:
            dist = 1e-16
        if n >= discard:
            s += math.log(dist / delta)
            count += 1
        # renormalize
        y = (x + (dx / dist) * delta) % 1.0
    return float(s / count) if count > 0 else float('nan')

# -------------------------
# Demo: compare baseline map (k=None) vs deep-zoom k in {1,2,3}
# -------------------------
if __name__ == "__main__":
    # Map parameters (pick a b value inside [6.1,10])
    params = {'b': 8.431, 'a': 0.9, 'c': 0.15}
    x0 = 0.321
    T = 20000
    T0 = 2000
    M = 200

    print("Custom map parameters:", params)
    print("Running baseline tests (k=None)...")
    res0, traj0 = time_vs_ensemble(custom_step, params, x0, None, T=T, T0=T0, M=M)
    print("Baseline results (time vs ensemble):")
    for kname, v in res0.items():
        print(f"  {kname}: time_avg={v['time_avg']:.6g}, ens_mean={v['ens_mean']:.6g},
ens_std={v['ens_std']:.6g}, ks_D={v['ks_D']:.6g}")

    # histogram L1 across different initials for baseline
    initials = [0.123, 0.2718, 0.419, 0.666, 0.801]
    trajs_baseline = [generate_trajectory(custom_step, s0, params, n_iters=T, discard=T0,
k=None) for s0 in initials]
    edges0, h0, l1_0 = hist_L1(trajs_baseline, nbins=300)
    print("Baseline histogram L1s (first 5):", l1_0[:5])

    ac0, slope0, intercept0 = autocorr_and_fit(traj0, maxlag=1000)
    lyap0 = lyapunov_fd(custom_step, params, x0=x0, k=None, T=20000, discard=2000)
    print("Baseline lyap approx:", lyap0, "autocorr slope:", slope0)

    # compare deep-zoom ks and hist L1 and lyap
    hk = None
    for kk in [1, 2, 3]:
        print(f"\nRunning deep-zoom with k={kk} ...")
        # time vs ensemble
        resk, trajk = time_vs_ensemble(custom_step, params, x0, kk, T=T, T0=T0, M=M)
        print(f"k={kk} results (time vs ensemble):")
        for kname, v in resk.items():
```

```
        print(f"  {kname}: time_avg={v['time_avg']:.6g}, ens_mean={v['ens_mean']:.6g},
ens_std={v['ens_std']:.6g}, ks_D={v['ks_D']:.6g}")

    # histogram L1
    trajs_k = [generate_trajectory(custom_step, s0, params, n_iters=T, discard=T0, k=kk)
for s0 in initials]
    edgesk, hk_list, l1_k = hist_L1(trajs_k, nbins=300)
    print(f"k={kk} histogram L1s (vs first init) first 5:", l1_k[:5])

    # autocorr and lyap
    ack, slopek, ik = autocorr_and_fit(trajk, maxlag=1000)
    lyapk = lyapunov_fd(custom_step, params, x0=x0, k=kk, T=20000, discard=2000)
    print(f"k={kk}: lyap approx: {lyapk:.6g}, autocorr slope: {slopek:.6g}")

    # store last hk for plotting
    hk = hk_list

# Optional: plot sample invariant densities (baseline vs last k)
plt.figure(figsize=(9, 4))
bc = 0.5 * (edges0[:-1] + edges0[1:])
plt.plot(bc, h0[0], label='baseline init1 (no deep-zoom)')
if hk is not None:
    plt.plot(bc, hk[0], label=f'k={kk} init1 (deep-zoom)')
plt.legend()
plt.title("Sample invariant densities (baseline vs deep-zoom)")
plt.xlabel("x")
plt.ylabel("density")
plt.tight_layout()
plt.show()
```

—----------

Adjusted SRQLM code based on your map and parameters.

```
import numpy as np
import matplotlib.pyplot as plt
import math

# ---- Map parameters ----
a = 0.9
c = 0.15
b = 8.0        # control parameter (change within [6.1, 10])
x0 = 0.321     # initial condition
```

```
params = {"a": a, "b": b, "c": c}

# ---- Your chaotic map F(x) ----
def mymap_step(x, p):
    x = float(x) % 1.0
    a = p["a"]; b = p["b"]; c = p["c"]
    term1 = a * np.sin(2 * np.pi * x)
    term2 = b * (x - 0.5)**3
    term3 = c * x
    return (term1 + term2 + term3) % 1.0

# ---- Generate trajectory ----
def generate_traj(x0, p, N=50000, discard=2000):
    x = float(x0) % 1.0
    traj = []
    for i in range(N + discard):
        x = mymap_step(x, p)
        if i >= discard:
            traj.append(x)
    return np.array(traj)

# ---- Benettin Lyapunov exponent ----
def lyapunov_benettin(x0, p, iters=50000, discard=5000, delta=1e-9):
    x = float(x0) % 1.0
    y = (x + delta) % 1.0
    ssum = 0.0; count = 0

    for n in range(iters):
        x = mymap_step(x, p)
        y = mymap_step(y, p)

        dx = ((y - x + 0.5) % 1.0) - 0.5
        dist = abs(dx) if abs(dx) > 1e-16 else 1e-16

        if n >= discard:
            ssum += math.log(dist / delta)
            count += 1

        # renormalize
        y = (x + (dx/dist)*delta) % 1.0

    return ssum / count

# ---- Run computations ----
print("Parameters:", params)
print("Generating trajectory...")
traj = generate_traj(x0, params, N=50000, discard=2000)
```

```
print("Computing Benettin Lyapunov (might take some time)...")
lyap = lyapunov_benettin(x0, params, iters=50000, discard=5000)
print("Estimated Lyapunov exponent:", lyap)

# ---- Numerical derivative ----
h = 1e-8
fprime_traj = [(mymap_step(x+h, params) - mymap_step(x-h, params)) / (2*h)
               for x in traj]
abs_fp = np.abs(fprime_traj)

p50, p90, p99 = np.percentile(abs_fp, [50, 90, 99])
print("Percentiles of |f'(x)|:", "p50=", p50, "p90=", p90, "p99=", p99)

# ---- Plot 1: Log separation ----
x1 = x0
x2 = x0 + 1e-9
sep = []
for i in range(2000):
    x1 = mymap_step(x1, params)
    x2 = mymap_step(x2, params)
    d = abs(((x2 - x1 + 0.5) % 1.0) - 0.5)
    sep.append(d if d > 1e-16 else 1e-16)

plt.figure(figsize=(7,4))
plt.plot(np.log(sep), lw=0.8)
plt.xlabel("Iteration")
plt.ylabel("log(|Δx|)")
plt.title(f"Log-separation of nearby trajectories\nMap b={b}, Lyapunov ≈ {lyap:.4f}")
plt.tight_layout()
plt.show()

# ---- Plot 2: Histogram of |f'(x)| ----
plt.figure(figsize=(7,4))
upper = np.percentile(abs_fp, 99.9)
plt.hist(abs_fp, bins=200, range=(0, upper), log=True)
plt.axvline(p50, linestyle="--", label=f"p50={p50:.3f}")
plt.axvline(p90, linestyle="--", label=f"p90={p90:.3f}")
plt.axvline(p99, linestyle="--", label=f"p99={p99:.3f}")
plt.xlabel("|f'(x)|")
plt.ylabel("Frequency (log scale)")
plt.legend()
plt.title("Distribution of |f'(x)| along trajectory")
plt.tight_layout()
plt.show()

# ---- Scan Lyapunov vs b ----
def lyap_vs_b(b_values, x0, p_template, iters=5000, discard=1000):
    L = []
```

```
    for bv in b_values:
        p = p_template.copy()
        p["b"] = bv
        L.append(lyapunov_benettin(x0, p, iters=iters, discard=discard))
    return np.array(L)

# Coarse Lyapunov curve
b_values = np.linspace(6.1, 10.0, 40)
print("Computing coarse Lyapunov vs b...")
lyap_curve = lyap_vs_b(b_values, x0, params, iters=5000, discard=1000)
print("Done.")

plt.figure(figsize=(8,3))
plt.plot(b_values, lyap_curve, marker='o', ms=4)
plt.axhline(0, color='k', linewidth=0.6)
plt.xlabel("b")
plt.ylabel("Lyapunov exponent (coarse)")
plt.title("Coarse Lyapunov vs b for chaotic map F(x)")
plt.tight_layout()
plt.show()
```